

CS-302 Artificial Intelligence

Experiment # 3

Solving a regression problem

Objective/s:

- Overview of regression
- Simple linear regression
- Multiple linear regression
- Example
- Lab Tasks

1. Overview of regression

Regression is a process of finding the correlations between dependent and independent variables. It helps in predicting the continuous variables such as prediction of Market Trends, prediction of House prices, etc.

The task of the Regression algorithm is to find the mapping function to map the input variable(x) to the continuous output variable(y).

Example: Suppose we want to do weather forecasting, so for this, we will use the Regression algorithm. In weather prediction, the model is trained on the past data, and once the training is completed, it can easily predict the weather for future days.

- Simple Linear Regression
- Multiple Linear Regression
- Polynomial Regression
- Support Vector Regression
- Decision Tree Regression
- Random Forest Regression

2. Simple linear regression

Simple linear regression is an approach for predicting a response using a single feature. It is assumed that the two variables are linearly related. Hence, we try to find a linear function that predicts the response value(y) as accurately as possible as a function of the feature or independent variable(x).

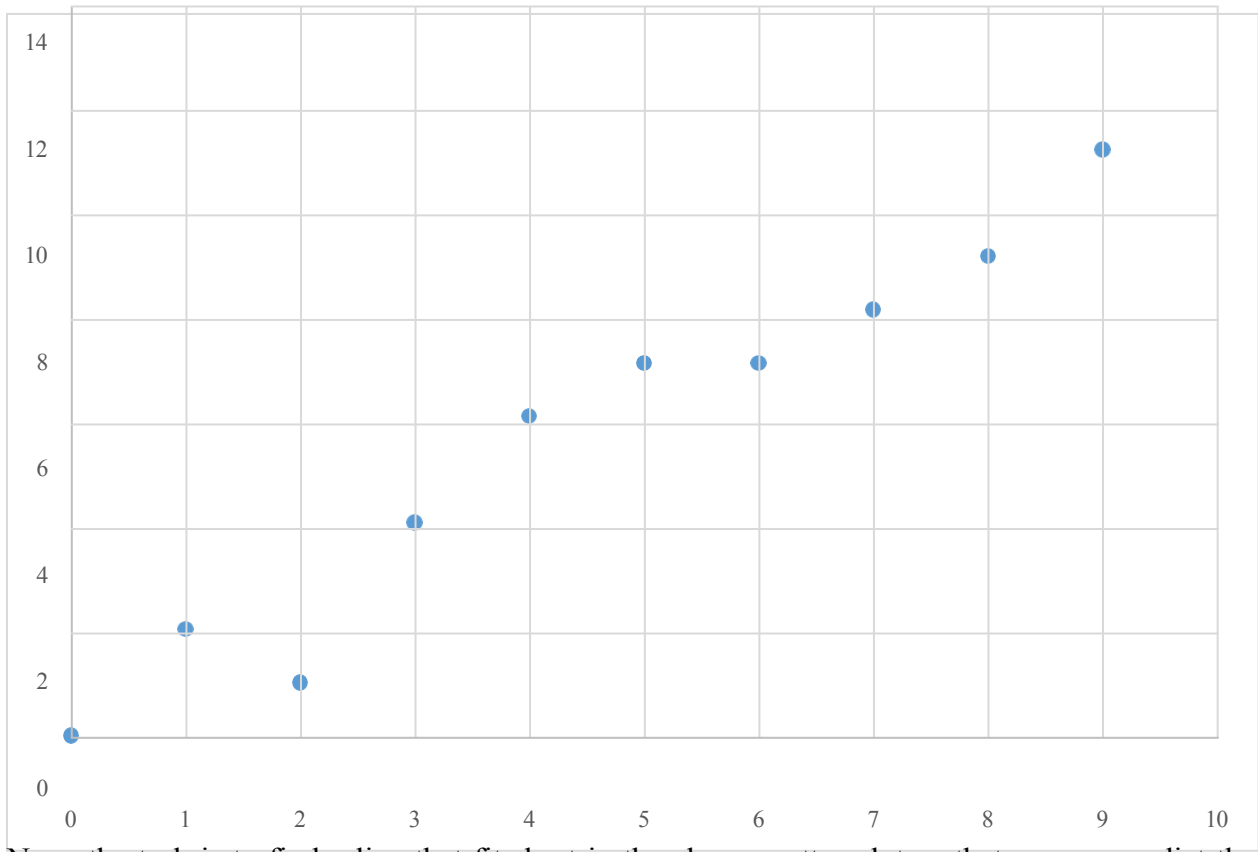
Let us consider a dataset where we have a value of response y for every feature x :

x	0	1	2	3	4	5	6	7	8	9
y	1	3	2	5	7	8	8	9	10	12

For generality, we define:

- x as feature vector, i.e $x = [x_1, x_2, \dots, x_n]$,
- y as response vector, i.e $y = [y_1, y_2, \dots, y_n]$

For n observations (in above example, $n=10$) a scatter plot of the above dataset looks like:



Now, the task is to find a line that fits best in the above scatter plot so that we can predict the response for any new feature values. (i.e. a value of x not present in a dataset). This line is called a regression line.

The equation of regression line is represented as:

$$h(x_i) = \beta_0 + \beta_1 x_i$$

Here,

- $h(x_i)$ represents the predicted response value for i th observation.
- β_0, β_1 are regression coefficients and represent y-intercept and slope of regression line respectively.

To create our model, we must “learn” or estimate the values of regression coefficients β_0 and β_1 . And once we’ve estimated these coefficients, we can use the model to predict responses.

In this lab experiment, we are going to use the principle of *Least Squares*.
Now consider:

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i = h(x_i) + \varepsilon_i \Rightarrow \varepsilon_i = y_i - h(x_i)$$

Here, ε_i is a residual error in *i*th observation. So, our aim is to minimize the total residual error. We define the squared error or cost function, J as:

$$J(\beta_0, \beta_1) = \frac{1}{2} \sum_{i=1}^n \varepsilon_i^2$$

$$0 \leq J \leq 2n$$

$$\sum_{i=1}^n$$

Our task is to find the value of β_0 and β_1 for which (β_0, β_1) is minimum.

Without going into the mathematical details, we present the result here:

$$\beta_1 = \frac{SS_{xy}}{SS_{xx}}$$

$$\beta_0 = \bar{y} - \beta_1 \bar{x}$$

where SS_{xy} is the sum of cross-deviations of y and x :

$$SS_{xy} = \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}) = \sum_{i=1}^n y_i x_i - n \bar{x} \bar{y}$$

and SS_{xx} is the sum of squared deviations of x :

$$SS_{xx} = \sum_{i=1}^n (x_i - \bar{x})^2 = \sum_{i=1}^n x_i^2 - n(\bar{x})^2$$

Note: The complete derivation for finding least squares estimates in simple linear regression can be found [here](#).

2.1. Code: Python implementation of above technique on our small dataset

```
import numpy as np
import matplotlib.pyplot as plt

def estimate_coef(x, y):
    # number of observations/points
    n = np.size(x)

    # mean of x and y vector
    m_x = np.mean(x)
    m_y = np.mean(y)

    # calculating cross-deviation and deviation about x
    SS_xy = np.sum(y*x) - n*m_y*m_x
    SS_xx = np.sum(x*x) - n*m_x*m_x

    # calculating regression coefficients
    b_1 = SS_xy / SS_xx
    b_0 = m_y - b_1*m_x

    return (b_0, b_1)

def plot_regression_line(x, y, b):
    # plotting the actual points as scatter plot
    plt.scatter(x, y, color = "m",
                marker = "o", s = 30)

    # predicted response vector
    y_pred = b[0] + b[1]*x

    # plotting the regression line
    plt.plot(x, y_pred, color = "g")

    # putting labels
    plt.xlabel('x')
    plt.ylabel('y')

    # function to show plot
    plt.show()

def main():
    # observations / data
    x = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
    y = np.array([1, 3, 2, 5, 7, 8, 8, 9, 10, 12])

    # estimating coefficients
    b = estimate_coef(x, y)
    print("Coefficients:b_0", b[0], "b_1 =", b[1])

    # plotting regression line
    plot_regression_line(x, y, b)

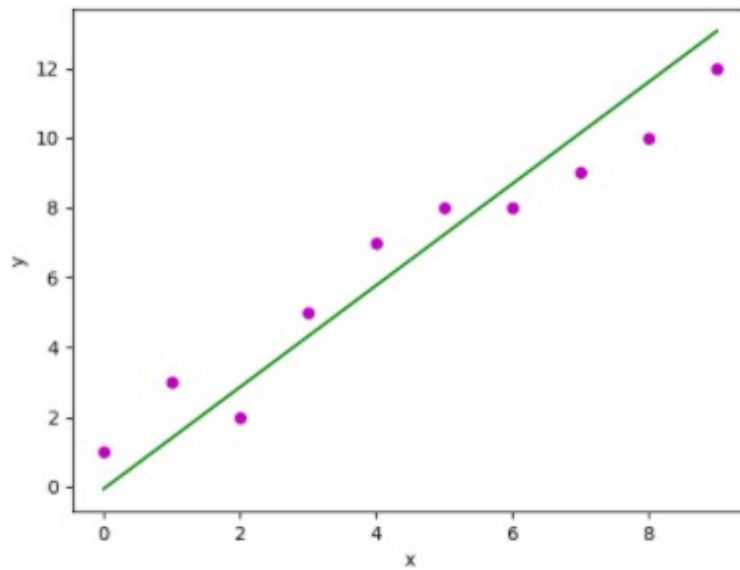
if __name__ == "__main__":
    main()
```

Output

Estimated coefficients:

$b_0 = -0.0586206896552$

$b_1 = 1.45747126437$



3. Tasks

1. Write the above mentioned program for different datasets.